

CSA-0619 CO1 AT1 ANSWERS

3. A hospital database stores patient records in an **unsorted list** because records are frequently added and updated. Doctors need to quickly search for patient details during emergencies.

a) Explain how sequential search works in this scenario.

b) Analyze best, worst, and average case performance.

c) Justify why sorting is not preferred here.

Q3. Hospital Database — Sequential Search Analysis

a) How Sequential Search Works in This Scenario

In a hospital database, patient records are stored in an **unsorted list** because new patients are admitted frequently, and existing records are updated often (medical history, prescriptions, test results, etc.). Maintaining sorted order after every insertion/update would be costly, so the list remains unordered.

Working of Sequential Search:

1. When a doctor needs to find a specific patient (e.g., by Patient ID or Name), the search starts from the **first record** in the list.
2. Each record is compared **one by one, in order**, with the target search key (patient ID/name).
3. If a match is found, the record's details (diagnosis, history, allergies, current medication, etc.) are returned immediately.
4. If the current record does not match, the search moves to the **next record** and repeats the comparison.
5. This continues until either:
 - a. The matching record is found (**successful search**), or
 - b. The entire list has been scanned without a match (**unsuccessful search**).

Algorithm (Pseudocode):

```
SEQUENTIAL_SEARCH(records[], n, target_id):  
  for i = 0 to n-1:  
    if records[i].id == target_id:  
      return records[i]    // record found  
  return NOT_FOUND        // record not found
```

Since there is no ordering assumption, this is the only search method directly applicable without preprocessing.

b) Best, Worst, and Average Case Performance

Case	Scenario	Number of Comparisons	Time Complexity
Best Case	The required patient record is the first element in the list	1 comparison	O(1)
Worst Case	The required patient record is the last element , or not present at all in the list	n comparisons	O(n)
Average Case	The record is equally likely to be found at any position in the list	$(n+1)/2$ comparisons	O(n)

Explanation:

- **Best Case — O(1):** Occurs when the patient's record happens to be at the very beginning of the list (e.g., a recently added record placed at the front). This is the fastest possible outcome but cannot be relied upon in general.
- **Worst Case — O(n):** Occurs when:
 - The target record is the last one in the list, or
 - The patient record does not exist (e.g., typo in ID, record not yet added).
In emergencies, this is a critical concern — for a hospital with a large number of patient records (large n), searching could take significant time.
- **Average Case — O(n):** On average, assuming the target is present and uniformly distributed across positions, approximately $n/2$ comparisons are needed. This is still **linear time**, meaning search time grows proportionally with the number of patient records.

c) Why Sorting Is Not Preferred Here

Sorting the patient list (to enable faster search methods like Binary Search — $O(\log n)$) is **not preferred** in this scenario for the following reasons:

1. **Frequent Insertions and Updates:**
Patient records are added constantly (new admissions, emergency cases) and updated frequently (treatment status, discharge, test results). Every insertion into a sorted list requires **shifting elements** to maintain order, costing **O(n)** per insertion — this overhead would occur far more often than searches in a highly dynamic environment.
2. **Sorting Cost is Expensive:**
Re-sorting the entire list after every update (or maintaining sorted order incrementally)

requires $O(n \log n)$ time for a full sort, or $O(n)$ for maintaining order on each insert — this cost outweighs the benefit of faster search, especially when the list changes more often than it is searched.

3. **Dynamic, Real-Time Nature of Hospital Data:**

Hospital databases are **highly dynamic** — records change in real time. Keeping a list perpetually sorted adds constant computational overhead that isn't justified for a system prioritizing **fast writes** (admissions/updates) over occasional reads.

4. **Better Alternatives Exist:**

Instead of sorting purely for search efficiency, hospitals typically use:

- a. **Hash tables / hash maps** (keyed by Patient ID) for $O(1)$ average-time lookups without needing sorted order.
- b. **Indexed database structures** (e.g., B-Trees in database management systems) that support both fast insertion **and** fast search — a better trade-off than a simple sorted array.

5. **Trade-off Summary:**

Approach	Search Time	Insertion/Update Time	Suitability
Unsorted List + Sequential Search	$O(n)$	$O(1)$	Good for frequent updates, simple to implement
Sorted List + Binary Search	$O(\log n)$	$O(n)$ (due to shifting)	Poor when updates are frequent
Hash Table	$O(1)$ average	$O(1)$ average	Best for this scenario overall

Conclusion

For a hospital database with frequent insertions and updates, **sequential search on an unsorted list** is a practical choice despite its $O(n)$ search time, because it avoids the high cost of maintaining sorted order. However, for real-world performance-critical systems (especially in emergencies where speed is vital), a **hash table** would be the ideal solution, offering $O(1)$ average search time while still supporting fast insertions and updates — combining the flexibility of unsorted structures with near-constant-time lookups.